ТОО ”Колледж Хекслет”

Лабораторная работа №3

Проектирование базы данных Internet Shop

Выполнено студентом:

Дубинин Кирилл

Практическое задание

Раздел RO4_3. Проектирование базы данных Internet Shop.

Цель: Переработать существующую структуру базы shop_db: выделить категорию товара в отдельную сущность, создать таблицу categories, связать её с products через внешний ключ и перенести существующие данные без потери товаров. 1. Посмотрите текущую структуру products Выполните: SELECT * FROM products ORDER BY id; Предположим, таблица содержит: id | name | price | category | in_stock ----------1 | Keyboard | 15000 | Accessories | true 2 | Mouse | 8000 | Accessories | true 3 | Monitor | 85000 | Monitors | false 4 | SSD 2TB | 65000 | Components | true Обратите внимание: значение Accessories повторяется.

Условие: RO4_3/lesson3/ЛР-3.pdf

Требования задания

• categories.id — PRIMARY KEY;

• categories.name — NOT NULL UNIQUE;

• products.id — PRIMARY KEY;

• products.name — NOT NULL;

• products.price — CHECK (price > 0);

• products.category_id — NOT NULL;

• products.category_id — FOREIGN KEY;

• сохранение всех существующих товаров;

• отсутствие старого текстового столбца category;

• получение названия категории через JOIN;

• работоспособность GET /api/products;

• работоспособность GET /api/products/:id.

Ход выполнения

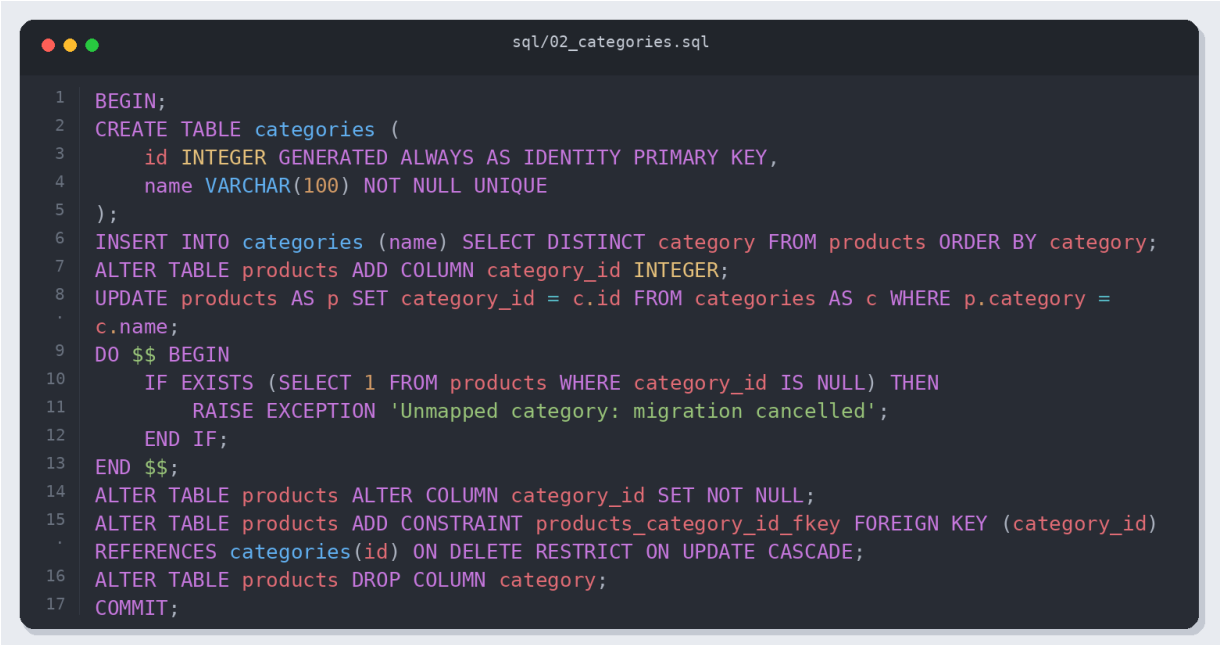
Категории выделены в таблицу categories с UNIQUE(name). В транзакции перенесены DISTINCT названия, заполнен category_id, проверено отсутствие NULL, добавлены NOT NULL и FOREIGN KEY. Старое поле category

удаляется после переноса. JOIN возвращает название категории вместе с categoryId. Будущие сущности: users(id PK, email UNIQUE), orders(id PK, user_id FK), order_items(order_id + product_id составной PK, quantity и price обязательны).

Проект: RO4_3/lesson3

Запуск: npm install; npm start. Для PostgreSQL параметры подключения задаются в .env по образцу .env.example. Порядок применения SQL и миграций приведён в README.md проекта.

sql/02_categories.sql



Листинг 1. sql/02_categories.sql

services/productService.js

services/productService.js

```
import { pool } from '../config/db.js';
export async function getAllProducts(query = {}) {
    const result = await pool.query(`SELECT p.id, p.name, p.price, p.category_id AS "categoryId", c.name AS category, p.in_stock AS "inStock" FROM products p JOIN categories c ON c.id = p.category_id
        WHERE ($1::text IS NULL OR c.name = $1)
        AND ($2::boolean IS NULL OR p.in_stock = $2)
        AND ($3::integer IS NULL OR p.price >= $3)
        AND ($4::integer IS NULL OR p.price <= $4) ORDER BY p.id`,
        [query.category ?? null, query.inStock ?? null, query.minPrice ?? null, query.maxPrice ?? null]);
    return result.rows;
}
export async function findProductById(id) {
    const result = await pool.query(`SELECT p.id, p.name, p.price, p.category_id AS "categoryId", c.name AS category, p.in_stock AS "inStock" FROM products p JOIN categories c ON c.id = p.category_id WHERE p.id = $1`, [id]);
    return result.rows[0];
}
export async function addProduct(data) {
    const result = await pool.query(`INSERT INTO products (name, price, category_id, in_stock)
        VALUES ($1, $2, $3, $4) RETURNING id`, [data.name, data.price, data.categoryId, data.inStock]);
    return findProductById(result.rows[0].id);
}
export async function updateProductById(id, changes) {
    const result = await pool.query(`UPDATE products SET name = COALESCE($1, name), price = COALESCE($2, price),
        category_id = COALESCE($3, category_id), in_stock = COALESCE($4, in_stock) WHERE id = $5 RETURNING id`,
        [changes.name ?? null, changes.price ?? null, changes.categoryId ?? null, changes.inStock ?? null, id]);
    return result.rows[0] ? findProductById(id) : null;
}
```

Листинг 2. services/productService.js (часть 1)

services/productService.js

```
export async function deleteProductById(id) {
    const result = await pool.query(`DELETE FROM products WHERE id = $1 RETURNING id, name, price, category_id, in_stock AS "inStock"`, [id]);
    return result.rows[0];
}
```

Листинг 3. services/productService.js (часть 2)

validators/productValidator.js

validators/productValidator.js

```
export function validateProductData(data, partial = false) {
    const errors = [];
    if (!data || typeof data !== 'object' || Array.isArray(data)) return ['JSON object required'];
    const fields = ['name', 'price', 'categoryId', 'inStock'];
    if (partial && Object.keys(data).length === 0) errors.push('At least one field is required');
    for (const field of Object.keys(data)) {
        if (!fields.includes(field)) errors.push(`Unknown field: ${field}`);
    }
    if (!partial || 'name' in data) {
        if (typeof data.name !== 'string' || !data.name.trim() || data.name.length > 150) errors.push('name is required (max 150)');
    }
    if (!partial || 'price' in data) {
        if (!Number.isSafeInteger(data.price) || data.price <= 0 || data.price > 2147483647) errors.push('price must be a positive integer');
    }
    if (!partial || 'categoryId' in data) { if (!Number.isSafeInteger(data.categoryId) || data.categoryId <= 0) errors.push('categoryId must be a positive integer'); }
    if (!partial || 'inStock' in data) {
        if (typeof data.inStock !== 'boolean') errors.push('inStock must be boolean');
    }

    return errors;
}
```

Листинг 4. validators/productValidator.js

Проверка результата

Проверено: 05.11.2026 13:38 (Алматы). Node.js v26.12.1.

Приведены фактические результаты проверки. Полный журнал: verification.json.

• Реальный PostgreSQL

• HTTP endpoints

• CRUD сохраняется в PostgreSQL

• UNIQUE и FOREIGN KEY; категории перенесены без потери товаров

HTTP-запросы

GET /api/products → 200

GET /api/products/1 → 200

GET /api/products/999999 → 404

GET /api/products/abc → 400

GET /api/products?category=Accessories&inStock=true&minPrice=10000&maxPrice=50000 → 200

GET /api/products?minPrice=abc → 400

GET /api/products?inStock=maybe → 400

GET /api/products?minPrice=500&maxPrice=100 → 400

POST /api/products → 403

POST /api/products → 400

POST /api/products → 201

PATCH /api/products/6 → 200

DELETE /api/products/6 → 200

GET /api/products/6 → 404

POST /api/products → 409

GET /api/unknown → 404

Примеры ответов

POST /api/products → 201


```
{
  "id": 6,
  "name": "Test SSD",
  "price": 65000,
  "categoryId": 1,
  "category": "Accessories",
  "inStock": true
```

}

PATCH /api/products/6 → 200

{
 "id": 6,
 "name": "Test SSD",
 "price": 42000,
 "categoryId": 1,
 "category": "Accessories",
 "inStock": false
}

DELETE /api/products/6 → 200

{
 "id": 6,
 "name": "Test SSD",
 "price": 42000,
 "category_id": 1,
 "inStock": false
}

Результат

Проверены обязательные операции и указанные ошибочные сценарии; результаты сохранены вместе с исходным кодом.